

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра “Програмна інженерія”

З В І Т

до лабораторної роботи №4

з дисципліни “Операційні системи”

з теми “КЕРУВАННЯ ЗОВНІШНІМИ ПРИСТРОЯМИ. НЕСТАНДАРТНІ
ПРИСТРОЇ”

Виконали:

здобувачі ПЗПІ-24-5

Кулик Є. В.

Клочков І. В.

Калашник А. М.

Прийняв:

ст. викл. Олійник О. О.

Харків 2026

ЛАБОРАТОРНА РОБОТА №4

1. Мета роботи

Навчитися практичному використанню функцій WINAPI для роботи з файлами.

2. Хід роботи

Вихідний код програми доступний для перегляду на GitHub: <https://github.com/IlliaKlochkov/OS/blob/lab4/Labs/lab4/lab4.cpp>

Програму розроблено у середовищі Visual Studio мовою C++ з використанням функцій WinAPI для роботи з файловою системою. Реалізовано моделювання поштової скриньки: зберігання повідомлень у бінарних файлах, керування мета-даними через окремий заголовковий файл, а також контроль цілісності на основі алгоритму CRC32.

2.1 Створення поштової скриньки

Функція ініціалізує структуру каталогів і записує початковий мета-файл із нульовими лічильниками та заданим максимальним розміром.

Наступний фрагмент коду реалізує створення поштової скриньки:

```
bool createMailbox(DWORD maxSizeBytes)
{
    if (!CreateDirectoryA("Mailbox", NULL) &&
        GetLastError() != ERROR_ALREADY_EXISTS)
    {
        printf("Помилка: неможливо створити каталог Mailbox\n");
        return false;
    }
    if (!CreateDirectoryA("Mailbox\\messages", NULL) &&
        GetLastError() != ERROR_ALREADY_EXISTS)
    {
        printf("Помилка: неможливо створити каталог messages\n");
        return false;
    }
    MailboxHeader hdr = {};
    hdr.msgCount = 0;
    hdr.totalSize = 0;
    hdr.maxSize = maxSizeBytes;
    hdr.crc32 = calcHeaderCRC(hdr);
    if (!writeHeader(hdr))
    {
        printf("Помилка: неможливо записати мета-файл\n");
    }
}
```

```

        return false;
    }
    g_mailboxCount++;
    printf("Поштову скриньку створено. Максимальний розмір: %u байт\n",
maxSizeBytes);
    return true;
}

```

Функція createMailbox() послідовно створює каталог Mailbox та підкаталог Mailbox\messages через CreateDirectoryA. Якщо каталог вже існує (ERROR_ALREADY_EXISTS) – помилка ігнорується. Ініціалізується структура MailboxHeader з нульовими лічильниками і переданим maxSizeBytes, підраховується CRC і заголовок записується у мета-файл. Глобальний лічильник g_mailboxCount збільшується на 1.

```

=====
1. Створення поштової скриньки (макс. 4096 байт)
=====
Поштову скриньку створено. Максимальний розмір: 4096 байт
Загальна кількість поштових скриньок: 1

```

Рисунок 1 – Результат виконання створення поштової скриньки

Після виклику createMailbox(4096) виведено повідомлення про успішне створення з максимальним розміром 4096 байтів та загальну кількість скриньок – 1.

2.2 Додавання листа

Кожне повідомлення зберігається як окремий бінарний файл із заголовком у 4 байти, де записується розмір повідомлення, та тілом самого повідомлення. Перед записом нового листа програма перевіряє, чи не перевищить нове повідомлення максимально допустимий розмір поштової скриньки.

Оскільки окремі листи можуть видалятися, індекс нового повідомлення не можна визначати лише за кількістю наявних листів. Для

цього реалізовано окрему функцію `findFreeMessageIndex()`, яка знаходить перший вільний номер файлу повідомлення.

```
DWORD findFreeMessageIndex()
{
    DWORD index = 0;

    while (true)
    {
        string path = getMessagePath(index);

        DWORD attrs = GetFileAttributesA(path.c_str());

        if (attrs == INVALID_FILE_ATTRIBUTES)
        {
            return index;
        }

        index++;
    }
}
```

Функція `findFreeMessageIndex()` послідовно перевіряє файли повідомлень `msg_0000.bin`, `msg_0001.bin`, `msg_0002.bin` тощо за допомогою функції `GetFileAttributesA`. Якщо файл з певним індексом не існує, цей індекс вважається вільним і використовується для створення нового листа. Такий підхід дозволяє коректно додавати повідомлення навіть після видалення окремих листів, не перезаписуючи вже існуючі файли.

Наступний фрагмент коду реалізує додавання листа:

```
bool addMessage(const string& body)
{
    MailboxHeader hdr;
    if (!readHeader(hdr))
    {
        printf("Помилка читання заголовку скриньки\n");
        return false;
    }

    DWORD msgSize = (DWORD)body.size();

    if (hdr.totalSize + msgSize > hdr.maxSize)
    {
        printf("Помилка: поштова скринька переповнена! (%u + %u > %u)\n",
            hdr.totalSize, msgSize, hdr.maxSize);
        return false;
    }

    // Знаходимо наступний вільний індекс (може бути «дірка» через
    видалення)
    DWORD newIndex = findFreeMessageIndex();

    string msgPath = getMessagePath(newIndex);
```

```

HANDLE hFile = CreateFileA(
    msgPath.c_str(),
    GENERIC_WRITE, 0, NULL,
    CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, NULL);

if (hFile == INVALID_HANDLE_VALUE)
{
    printf("Помилка: неможливо створити файл повідомлення\n");
    return false;
}

// Спочатку записуємо розмір повідомлення (4 байти)
DWORD bytesWritten = 0;
WriteFile(hFile, &msgSize, sizeof(msgSize), &bytesWritten, NULL);

// Потім саме тіло
WriteFile(hFile, body.c_str(), msgSize, &bytesWritten, NULL);
CloseHandle(hFile);

hdr.msgCount++;
hdr.totalSize += msgSize;
hdr.crc32 = calcHeaderCRC(hdr);

if (!writeHeader(hdr))
{
    printf("Помилка оновлення заголовку\n");
    return false;
}

printf("Лист #%u додано (%u байт)\n", newIndex, msgSize);
return true;
}

```

Функція `addMessage()` зчитує поточний заголовок і перевіряє умову $\text{totalSize} + \text{msgSize} > \text{maxSize}$. Якщо додавання нового листа не перевищує максимальний розмір поштової скриньки, програма визначає індекс нового повідомлення за допомогою функції `findFreeMessageIndex()`.

Далі формується шлях до файлу повідомлення через `getMessagePath()`. Файл створюється через `CreateFileA` з `CREATE_ALWAYS`. Спочатку у файл записується 4-байтовий розмір повідомлення, потім тіло листа. Після запису оновлюється заголовок: `msgCount++` і `totalSize += msgSize`, а CRC32 перераховується.

```

=====
2. Додавання листів
=====
Лист #0 додано (78 байт)
Лист #1 додано (99 байт)
Лист #2 додано (67 байт)
Лист #3 додано (46 байт)

```

Рисунок 2 – Результат виконання додавання листів

Додавали чотири листи. Програма вивела підтвердження для кожного: лист #0 – 78 байт, #1 – 99 байт, #2 – 67 байт, #3 – 46 байт.

2.3 Виведення інформації про скриньку

Функція зчитує заголовок і виводить поточний стан скриньки: кількість листів, загальний та максимальний розмір, значення CRC32.

Наступний фрагмент коду реалізує виведення стану поштової скриньки:

```

void printMailboxInfo()
{
    MailboxHeader hdr;
    if (!readHeader(hdr))
    {
        printf("Помилка читання заголовку\n");
        return;
    }
    printf(" Кількість листів : %u\n", hdr.msgCount);
    printf(" Загальний розмір : %u байт\n", hdr.totalSize);
    printf(" Максимальний розмір: %u байт\n", hdr.maxSize);
    printf(" CRC32 заголовку : 0x%08X\n", hdr.crc32);
}

```

Функція printMailboxInfo() зчитує заголовок і виводить чотири поля: кількість листів, загальний та максимальний розмір у байтах, а також значення CRC32 у шістнадцятковому форматі.

```

=====
3. Стан скриньки після додавання
=====
Кількість листів : 4
Загальний розмір : 290 байт
Максимальний розмір: 4096 байт
CRC32 заголовку : 0x79D130FF

```

Рисунок 3 – Стан скриньки після додавання листів

Після додавання чотирьох листів: кількість – 4, загальний розмір – 290 байтів, максимальний – 4096 байтів, CRC32 – 0x79D130FF.

2.4 Читання листа

Читання реалізовано з можливістю видалення повідомлення після прочитання. Це дозволяє імітувати отримання листа з поштової скриньки: повідомлення можна або просто переглянути, або прочитати й одразу видалити.

Наступний фрагмент коду реалізує читання листа:

```
bool readMessage(DWORD index, bool deleteAfterRead)
{
    string msgPath = getMessagePath(index);
    HANDLE hFile = CreateFileA(
        msgPath.c_str(),
        deleteAfterRead ? (GENERIC_READ | GENERIC_WRITE) : GENERIC_READ,
        0, NULL,
        OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);
    if (hFile == INVALID_HANDLE_VALUE)
    {
        printf("Помилка: лист #%u не знайдено\n", index);
        return false;
    }
    DWORD msgSize = 0, bytesRead = 0;
    ReadFile(hFile, &msgSize, sizeof(msgSize), &bytesRead, NULL);
    string body(msgSize, '\0');
    ReadFile(hFile, &body[0], msgSize, &bytesRead, NULL);
    CloseHandle(hFile);
    printf("--- Лист   #%u   (%u байт)   ---\n%s\n", index, msgSize,
body.c_str());
    if (deleteAfterRead)
    {
        if (!DeleteFileA(msgPath.c_str()))
        {
            printf("Попередження: не вдалось видалити файл листа\n");
            return false;
        }
        MailboxHeader hdr;
        if (readHeader(hdr))
        {
            if (hdr.msgCount > 0) hdr.msgCount--;
            if (hdr.totalSize >= msgSize) hdr.totalSize -= msgSize;
            hdr.crc32 = calcHeaderCRC(hdr);
            writeHeader(hdr);
        }
        printf("(Лист #%u видалено після читання)\n", index);
    }
    return true;
}
```

Функція `readMessage()` відкриває файл повідомлення за індексом. Якщо параметр `deleteAfterRead` встановлено – файл відкривається з правами `GENERIC_READ | GENERIC_WRITE`. Спочатку зчитується 4-байтовий розмір, потім тіло у рядок. Якщо `deleteAfterRead` – файл видаляється через `DeleteFileA`, а заголовок оновлюється: `msgCount--` і `totalSize -= msgSize`.

```
=====
4. Читання листа #0 (без видалення)
=====
--- Лист #0 (78 байт) ---
Привіт! Це перший лист у поштовій скриньці.

=====
5. Читання листа #1 (з видаленням)
=====
--- Лист #1 (99 байт) ---
Другий лист: WinAPI дозволяє працювати з файлами напряму.
(Лист #1 видалено після читання)
```

Рисунок 4 – Результат виконання читання листів

Лист #0 прочитано без видалення – виведено тіло повідомлення, файл збережено. Лист #1 прочитано з видаленням – виведено тіло та підтвердження видалення.

2.5 Стан скриньки після читання з видаленням

Після читання листа #1 з видаленням стан скриньки змінився. Повторний виклик `printMailboxInfo()` показує оновлені лічильники.

```
=====
6. Стан скриньки після читання з видаленням
=====
Кількість листів : 3
Загальний розмір : 191 байт
Максимальний розмір: 4096 байт
CRC32 заголовку : 0x1FBB73E4
```

Рисунок 5 – Стан скриньки після читання з видаленням

Кількість листів зменшилась до 3, загальний розмір – 191 байт, CRC32 оновлено до 0x1FBB73E4.

2.6 Видалення конкретного листа

Функція видаляє файл повідомлення та оновлює заголовок скриньки.

Наступний фрагмент коду реалізує видалення конкретного листа:

```
bool deleteMessage(DWORD index)
{
    string msgPath = getMessagePath(index);
    HANDLE hFile = CreateFileA(
        msgPath.c_str(),
        GENERIC_READ, FILE_SHARE_READ, NULL,
        OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);
    DWORD msgSize = 0;
    if (hFile != INVALID_HANDLE_VALUE)
    {
        DWORD br = 0;
        ReadFile(hFile, &msgSize, sizeof(msgSize), &br, NULL);
        CloseHandle(hFile);
    }
    else
    {
        printf("Помилка: лист #%u не знайдено\n", index);
        return false;
    }
    if (!DeleteFileA(msgPath.c_str()))
    {
        printf("Помилка видалення листа #%u\n", index);
        return false;
    }
    MailboxHeader hdr;
    if (readHeader(hdr))
    {
        if (hdr.msgCount > 0) hdr.msgCount--;
        if (hdr.totalSize >= msgSize) hdr.totalSize -= msgSize;
        hdr.crc32 = calcHeaderCRC(hdr);
        writeHeader(hdr);
    }
    printf("Лист #%u видалено\n", index);
    return true;
}
```

Функція deleteMessage() відкриває файл для зчитування розміру (щоб коректно оновити totalSize), після чого видаляє файл через DeleteFileA. Якщо відкрити файл не вдалось – повертає false одразу. Після видалення оновлює заголовок: msgCount-- і totalSize -= msgSize, перераховує CRC.

```
=====
7. Видалення листа #2
=====
Лист #2 видалено
Кількість листів: 2
```

Рисунок 6 – Результат виконання видалення листа #2

Після видалення листа #2 функція `getMessageCount()` повернула 2 – кількість листів зменшилась на 1.

2.7 Контроль цілісності (CRC32)

Для контролю цілісності реалізовано алгоритм CRC32. Таблиця поліномів будується один раз і кешується у глобальному масиві.

Наступний фрагмент коду реалізує функції підрахунку CRC32:

```
static DWORD crc32Table[256];
static bool  crc32TableReady = false;

void buildCRC32Table()
{
    if (crc32TableReady) return;
    for (DWORD i = 0; i < 256; i++)
    {
        DWORD c = i;
        for (int j = 0; j < 8; j++)
            c = (c & 1) ? (0xEDB88320u ^ (c >> 1)) : (c >> 1);
        crc32Table[i] = c;
    }
    crc32TableReady = true;
}

DWORD calcCRC32(const BYTE* data, DWORD len)
{
    buildCRC32Table();
    DWORD crc = 0xFFFFFFFFu;
    for (DWORD i = 0; i < len; i++)
        crc = crc32Table[(crc ^ data[i]) & 0xFF] ^ (crc >> 8);
    return crc ^ 0xFFFFFFFFu;
}
```

Функція `buildCRC32Table()` заповнює масив `crc32Table[256]` за стандартним поліномом `0xEDB88320`. Перевірка `crc32TableReady` гарантує, що таблиця будується лише раз. Функція `calcCRC32()` обчислює

контрольну суму методом побітового XOR із таблицею та повертає інвертований результат.

Наступний фрагмент коду реалізує перевірку цілісності поштової скриньки:

```
bool checkIntegrity()
{
    MailboxHeader hdr;
    if (!readHeader(hdr))
    {
        printf("Помилка: не вдалось прочитати заголовок для перевірки
цілісності\n");
        return false;
    }
    DWORD expected = calcHeaderCRC(hdr);
    if (expected != hdr.crc32)
    {
        printf("ЦІЛІСНІСТЬ ПОРУШЕНА! CRC заголовку: збережений=0x%08X,
розрахований=0x%08X\n",
            hdr.crc32, expected);
        return false;
    }
    WIN32_FIND_DATA fd;
    HANDLE hFind = FindFirstFileA("Mailbox\\messages\\msg_*.bin", &fd);
    DWORD realCount = 0;
    DWORD realSize = 0;
    if (hFind != INVALID_HANDLE_VALUE)
    {
        do
        {
            string fullPath = string("Mailbox\\messages\\") +
fd.cFileName;

            HANDLE hMsg = CreateFileA(
                fullPath.c_str(), GENERIC_READ, FILE_SHARE_READ,
                NULL, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);
            if (hMsg != INVALID_HANDLE_VALUE)
            {
                DWORD msgSize = 0, br = 0;
                ReadFile(hMsg, &msgSize, sizeof(msgSize), &br, NULL);
                CloseHandle(hMsg);
                realCount++;
                realSize += msgSize;
            }
        } while (FindNextFileA(hFind, &fd));
        FindClose(hFind);
    }
    bool ok = (realCount == hdr.msgCount) && (realSize == hdr.totalSize);
    if (ok)
        printf("Цілісність OK: CRC=0x%08X, листів=%u, розмір=%u байт\n",
            hdr.crc32, hdr.msgCount, hdr.totalSize);
    else
        printf("НЕВІДПОВІДНІСТЬ! Заголовок каже: листів=%u, розмір=%u; "
            "Реально: листів=%u, розмір=%u\n",
            hdr.msgCount, hdr.totalSize, realCount, realSize);
    return ok;
}
```

Функція `checkIntegrity()` виконує дворівневу перевірку. Спочатку порівнює збережений `crc32` у заголовку з повторно обрахованим. Далі обходить усі файли повідомлень через `FindFirstFileA`, підраховує реальну кількість файлів і сумарний розмір та порівнює з даними з заголовка.

```
=====
8. Перевірка цілісності (CRC32)
=====
Цілісність OK: CRC=0x70377F5F, листів=2, розмір=124 байт
```

```
=====
10. Фінальна перевірка цілісності
=====
Цілісність OK: CRC=0x67F3651F, листів=0, розмір=0 байт
```

Рисунок 7 – Результат виконання перевірки цілісності

Перевірку запускали двічі: після видалення листа #2 – «Цілісність OK: CRC=0x70377F5F, листів=2, розмір=124 байт»; після видалення всіх листів – «Цілісність OK: CRC=0x67F3651F, листів=0, розмір=0 байт».

2.8 Видалення всіх листів

Наступний фрагмент коду реалізує видалення всіх листів:

```
bool deleteAllMessages()
{
    MailboxHeader hdr;
    if (!readHeader(hdr)) return false;
    WIN32_FIND_DATA fd;
    HANDLE hFind = FindFirstFileA("Mailbox\\messages\\msg_*.bin", &fd);
    if (hFind == INVALID_HANDLE_VALUE)
    {
        printf("Листів немає – скринька вже порожня\n");
        return true;
    }
    DWORD deleted = 0;
    do
    {
        string fullPath = string("Mailbox\\messages\\") + fd.cFileName;
        if (DeleteFileA(fullPath.c_str()))
            deleted++;
    } while (FindNextFileA(hFind, &fd));
    FindClose(hFind);
    hdr.msgCount = 0;
    hdr.totalSize = 0;
    hdr.crc32 = calcHeaderCRC(hdr);
    writeHeader(hdr);
}
```

```
printf("Видалено %u листів\n", deleted);
return true;
}
```

Функція `deleteAllMessages()` використовує `FindFirstFileA` і `FindNextFileA` для перебору всіх файлів за маскою `msg_*.bin`. Кожен знайдений файл видаляється через `DeleteFileA`. Після завершення обходу заголовок обнуляється: `msgCount = 0`, `totalSize = 0`, CRC перераховується.

```
=====
9. Видалення всіх листів
=====
Видалено 2 листів
Кількість листів: 0
```

Рисунок 8 – Результат виконання видалення всіх листів

Після виклику `deleteAllMessages()` виведено «Видалено 2 листів», `getMessageCount()` повернув 0 – каталог `messages` порожній.

2.9 Допоміжні функції та операції з мета-файлом

Реалізовано допоміжні функції для ініціалізації консолі, виведення роздільників і формування шляху до файлу повідомлення.

Наступний фрагмент коду реалізує допоміжні функції:

```
void setupLocale()
{
    setlocale(LC_ALL, "Ukrainian");
    SetConsoleOutputCP(1251);
    SetConsoleCP(1251);
}

void printSeparator(const char* title)
{
    printf("\n=====\\n");
    printf("%s\\n", title);
    printf("=====\\n");
}

string getMessagePath(DWORD index)
{
    ostringstream oss;
    oss << "Mailbox\\messages\\msg_" << setw(4) << setfill('0') << index
    << ".bin";
    return oss.str();
}
```

Функція `setupLocale()` налаштовує локаль на `Ukrainian` та встановлює кодову сторінку консолі `1251`. Функція `printSeparator()` виводить заголовок між рядками символів «=». Функція `getMessagePath()` формує шлях до файлу повідомлення у форматі `Mailbox\messages\msg_NNNN.bin` з індексом доповненим нулями до 4 цифр.

Заголовок поштової скриньки зберігається у бінарному файлі `mailbox_meta.bin`. Для читання і запису реалізовано окремі функції.

Наступний фрагмент коду реалізує операції з мета-файлом:

```
bool readHeader(MailboxHeader& hdr)
{
    HANDLE hFile = CreateFileA(
        "Mailbox\\mailbox_meta.bin",
        GENERIC_READ, FILE_SHARE_READ, NULL,
        OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);
    if (hFile == INVALID_HANDLE_VALUE)
        return false;
    DWORD bytesRead = 0;
    BOOL ok = ReadFile(hFile, &hdr, sizeof(hdr), &bytesRead, NULL);
    CloseHandle(hFile);
    return ok && (bytesRead == sizeof(hdr));
}

bool writeHeader(const MailboxHeader& hdr)
{
    HANDLE hFile = CreateFileA(
        "Mailbox\\mailbox_meta.bin",
        GENERIC_WRITE, 0, NULL,
        CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, NULL);
    if (hFile == INVALID_HANDLE_VALUE)
        return false;
    DWORD bytesWritten = 0;
    BOOL ok = WriteFile(hFile, &hdr, sizeof(hdr), &bytesWritten, NULL);
    CloseHandle(hFile);
    return ok && (bytesWritten == sizeof(hdr));
}

DWORD calcHeaderCRC(const MailboxHeader& hdr)
{
    return calcCRC32((const BYTE*)&hdr, 12);
}
```

Функція `readHeader()` відкриває мета-файл з прапорцем `OPEN_EXISTING` і зчитує рівно `sizeof(MailboxHeader)` байтів. Функція `writeHeader()` відкриває файл з `CREATE_ALWAYS` і записує структуру заголовка. Функція `calcHeaderCRC()` обраховує CRC32 перших 12 байтів заголовка – полів `msgCount`, `totalSize` та `maxSize`, виключаючи поле `crc32`.

Наступний фрагмент коду реалізує функції отримання кількості листів та скриньок:

```
DWORD getMessageCount()
{
    MailboxHeader hdr;
    if (!readHeader(hdr)) return 0;
    return hdr.msgCount;
}

int getTotalMailboxCount()
{
    return g_mailboxCount;
}
```

Функція `getMessageCount()` зчитує поточний заголовок і повертає поле `msgCount`. Функція `getTotalMailboxCount()` повертає глобальну змінну `g_mailboxCount`, яка збільшується при кожному успішному виклику `createMailbox()`.

Висновки

У ході лабораторної роботи було реалізовано моделювання поштової скриньки засобами WinAPI на мові C++. Повідомлення зберігаються як окремі бінарні файли з 4-байтовим розміром і тілом листа, а мета-дані – у файлі `mailbox_meta.bin`. Реалізовано створення скриньки, додавання, читання з видаленням і без, видалення одного або всіх листів, а також визначення кількості листів і скриньок.

Під час додавання нового листа використовується пошук першого вільного індексу, тому після видалення повідомлень нові файли не перезаписують уже наявні. Для контролю цілісності реалізовано CRC32, а функція `checkIntegrity()` звіряє дані мета-файлу з фактичною кількістю та розміром повідомлень.

Усі основні операції з файлами й каталогами виконуються через функції WinAPI: `CreateDirectory`, `CreateFile`, `ReadFile`, `WriteFile`, `DeleteFile`, `FindFirstFile`, `FindNextFile` і `CloseHandle`.

Контрольні питання

1. Які прапорці необхідно використовувати для функції CreateFile для створення нового і відкриття існуючого файлу?

Для створення нового файлу використовують прапорець CREATE_NEW (якщо файл вже існує – помилка) або CREATE_ALWAYS (перезаписує існуючий). Для відкриття існуючого – OPEN_EXISTING (якщо файл не існує – помилка) або OPEN_ALWAYS (відкриває або створює).

2. Задайте прапорці для функції CreateFile для дозволу введення-виведення з файлу іншими програмами.

Для дозволу паралельного доступу інших процесів використовують параметр dwShareMode зі значенням FILE_SHARE_READ | FILE_SHARE_WRITE. Наприклад: CreateFileA(path, GENERIC_READ, FILE_SHARE_READ | FILE_SHARE_WRITE, NULL, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL).

3. Задайте прапорці для функції CreateFile для файлу, який спочатку читається, а потім модифікується.

У параметрі dwDesiredAccess слід вказати GENERIC_READ | GENERIC_WRITE, щоб відкрити файл і для читання, і для запису. Прапорець dwCreationDisposition – OPEN_EXISTING, якщо файл повинен існувати. Саме так реалізовано читання з видаленням у функції readMessage().

4. Яка функція використовується для визначення розміру файлу?

Для отримання розміру файлу використовують функцію `GetFileSize(hFile, NULL)`, яка повертає молодші 32 біти розміру. Для файлів більше 4 ГБ – `GetFileSizeEx(hFile, &liSize)`, де `liSize` — структура `LARGE_INTEGER`.

5. Які параметри необхідно визначити для файлу, розмір якого менше $2^{32} - 1$?

Достатньо використовувати `GetFileSize()` з другим параметром `NULL` – функція поверне розмір як значення `DWORD`. Якщо розмір точно менший за 4 294 967 295 байтів, старші 32 біти не потрібні і `lpFileSizeHigh` можна не передавати.

6. За допомогою якої функції можна зміститися в файлі на задану величину?

Для позиціонування покажчика у файлі використовують `SetFilePointer(hFile, lDistanceToMove, NULL, dwMoveMethod)`, де `dwMoveMethod` може приймати значення `FILE_BEGIN` (від початку), `FILE_CURRENT` (від поточної позиції) або `FILE_END` (від кінця файлу). Для зміщення більш ніж на 2 ГБ використовують `SetFilePointerEx()`.

7. Як задати відносно чого виконується зміщення?

Відносна точка задається параметром `dwMoveMethod` функції `SetFilePointer`: `FILE_BEGIN` – зміщення від початку файлу (абсолютна позиція), `FILE_CURRENT` – зміщення відносно поточної позиції покажчика, `FILE_END` – зміщення відносно кінця файлу (від'ємні значення – всередину файлу).

8. Які засоби контролю цілісності ви знаєте?

Найпоширеніші засоби контролю цілісності – CRC32 (використано в даній роботі), MD5, SHA-1, SHA-256. CRC32 швидкий і простий, але не криптографічний – призначений для виявлення випадкових помилок. MD5 і SHA-сімейство – криптографічні хеш-функції, стійкіші до навмисних підробок, але повільніші. Також застосовують контрольні суми Adler-32, xxHash для високопродуктивних сценаріїв.